Ombygging: API-et returnerer spillet ved oppretting og endring

POST svarer 201 med det nye spillet, PUT /api/games/{id} svarer 200 med det endrede, og adminsiden oppdaterer bare det ene spillet i lista i stedet for å hente alle på nytt

Dokumentet beskriver to ombygginger av AK2. Den første gjør at `POST /api/games` sender det nye spillet tilbake, slik oppgave 4 krever. Den andre legger til `PUT /api/games/{id}`, som endrer hele spillet og også sender det tilbake. I begge tilfellene bruker `/admin/games` svaret til å oppdatere lista lokalt, uten et ekstra kall for å hente alle spillene.

Kravene fra oppgave 4

Metode og rute	Krav i tabellen	Før	Etter
`POST /api/games`	201 med det nye spillet, 400 ved ugyldige data	204 uten innhold, 400	201 med spillet og `Location`, 400
`PUT /api/games/{id}`	204 eller 200, 400 ved ugyldige data, 404 når id-en ikke finnes	Finnes ikke. Bare `/category` og `/rating`	200 med spillet, 400, 404

I tillegg sier kravene at «listen skal vise endringene etter oppretting, endring og sletting uten at brukeren laster siden på nytt», og at valideringen minst skal sjekke at navn er påkrevd, at MinPlayers er minst 1, og at MaxPlayers ikke er lavere enn MinPlayers. Den siste delen av valideringen ble ikke sjekket på serveren før, og er lagt til i del 2.

Filer som endres

Fil	Del 1: oppretting	Del 2: endring
`Data/GamesStorageAndQuery.cs`	`CreateGameAsync` returnerer spillet	Ny `UpdateGameAsync`
`ServerServices/ServerGamesService.cs`	`CreateGameAsync` returnerer `GameDTO`	Felles `ValidateGame`, ny `UpdateGameAsync`
`Program.cs`	`Results.Created(...)`	Nytt `MapPut("{id:int}")`
`Client/GameApiClient.cs`	`CreateGameAsync` leser spillet fra svaret	Ny `UpdateGameAsync`
`Client/UpdateGameRequest.cs`		Ny fil
`Components/Pages/AdminGames.razor`	Legger det nye spillet inn i lista	Endringsskjema, bytter ut ett spill i lista

Del 1: Oppretting returnerer det nye spillet

Utgangspunktet

Før ombyggingen returnerte ingen av leddene noe. Lagringen la spillet i lista og ga id-en med `nextId++`, men verken spillet eller id-en ble sendt tilbake. Endepunktet svarte `204 No Content`, og adminsiden visste derfor ikke hvilket spill som var laget. For å vise det nye spillet måtte siden hente alle spillene på nytt med `LoadGames()`.

Kallene før og etter

```
FØR: to runder mot API-et
AdminGames → POST /api/games   → ServerGamesService → GamesStorageAndQuery
           ← 204 No Content    (ingen data)
AdminGames → GET  /api/games   → ServerGamesService → GamesStorageAndQuery
           ← 200 + alle spill  → tabellen tegnes på nytt

ETTER: én runde
AdminGames → POST /api/games   → ServerGamesService → GamesStorageAndQuery
           ← 201 + det nye spillet (med id)  → legges bakerst i lista
```

Steg 1: Lagringen returnerer spillet

Spillet lagres i en variabel før det legges i lista, slik at det kan returneres. Returtypen endres fra `Task` til `Task<BoardGame>`. Det er her id-en blir til, så det er her den må sendes tilbake fra.

Data/GamesStorageAndQuery.cs

```
// Create. Returns the stored game, so the caller gets the id the storage handed out
public async Task<BoardGame> CreateGameAsync(string name, int minPlayers, int maxPlayers,
    string category)
{
    var game = new BoardGame(name, minPlayers, maxPlayers, category)
    {
        Id = nextId++
    };

    boardGames.Add(game);
    return game;
}
```

`DummyGames` kaller også `CreateGameAsync`. Den trenger ingen endring, fordi en returverdi som ikke brukes, bare ignoreres.

Steg 2: Servicen gjør spillet om til en DTO

Servicen tar imot `BoardGame` fra lagringen og gjør det om med `ToDto`, som allerede fantes. Slik sendes bare feltene i `GameDTO` ut av API-et, på samme måte som ved GET.

ServerServices/ServerGamesService.cs

```
public async Task<GameDTO> CreateGameAsync(CreateGameRequest request)
{
    // Adds validation
    ValidateGame(request);

    // If all validation passes, then its safe to call the data layer
    var game = await query.CreateGameAsync(request.Name, request.MinPlayers, request.MaxPlayers,
     request.Category);

    // Returns the stored game as a DTO, including the new id
    return ToDto(game);
}
```

Steg 3: Endepunktet svarer 201 Created

`Results.Created` gir statuskoden 201, legger spillet i svaret som JSON og setter `Location`-headeren til adressen der spillet kan hentes. 400 ved ugyldige data er uendret, fordi `ArgumentException` fortsatt fanges.

Program.cs

```
gamesApi.MapPost("", async (CreateGameRequest request, ServerGamesService gameService) =>
{
    try
    {
        var game = await gameService.CreateGameAsync(request);

        // 201 Created with the new game, and the address of the game in the Location header
        return Results.Created($"/api/games/{game.Id}", game);
    }

    catch (ArgumentException exception)
    {
        return Results.BadRequest(new
        {
            error = exception.Message
        });
```

Steg 4: GameApiClient leser spillet fra svaret

Klienten leser `GameDTO` fra svaret og gjør det om til `BoardGame` med `ToBoardGame`, som `GetGameByIdAsync` allerede bruker. Hvis svaret mot formodning er tomt, kastes `HttpRequestException`. Da havner det i den samme `catch`-blokken på siden som andre feil fra API-et.

Client/GameApiClient.cs

```
// Returns the game the API created, with the id the server handed out
public async Task<BoardGame> CreateGameAsync(CreateGameRequest request)
{
    var response = await http.PostAsJsonAsync("api/games", request);
    response.EnsureSuccessStatusCode();

    var dto = await response.Content.ReadFromJsonAsync<GameDTO>()
        ?? throw new HttpRequestException("API-et returnerte ikke spillet");

    return ToBoardGame(dto);
}
```

Steg 5: Adminsiden legger spillet inn i lista

I stedet for `await LoadGames()` legges det returnerte spillet bakerst i lista. `games` er en `IReadOnlyList<BoardGame>`, som ikke har `Add`. Uttrykket `[.. games ?? [], createdGame]` lager derfor en ny liste med de eksisterende spillene og det nye bakerst. Når `games` får en ny verdi, tegner Blazor tabellen på nytt etter at hendelsen er ferdig.

```
Components/Pages/AdminGames.razor

private async Task Create()
{
    try
    {
        var createdGame = await ApiClient.CreateGameAsync(NewGame);

        // Resets user window
        NewGame = new CreateGameRequest();

        // Adds the returned game to the list, instead of fetching all games again
        games = [.. games ?? [], createdGame];
        error = null;
    }

    catch
    {
        error = "Kunne ikke opprette spillet";
    }
```

Fordelen med endringen

- **Ett kall i stedet for to.** Oppretting gir én runde mot API-et. Før gikk det en POST og deretter en GET av hele samlingen.
- **Mindre data over nettverket.** Bare det ene spillet sendes tilbake, ikke alle. Forskjellen vokser med størrelsen på samlingen.
- **Klienten får id-en.** Id-en lages i lagringen. Nå vet siden hvilken id spillet fikk, og kan bruke den med en gang, for eksempel til en lenke til `/games/{id}`, eller til å endre og slette spillet.
- **Svaret viser hva som faktisk ble lagret.** Hvis serveren senere fyller inn standardverdier eller endrer noe, får siden de lagrede verdiene, ikke bare det brukeren skrev.
- **Ingen «laster»-blink.** Tabellen bytter ikke til «Laster spill...» mens alt hentes på nytt. Det nye spillet dukker bare opp nederst.
- **I tråd med REST.** `201 Created` med ressursen og `Location` er den vanlige måten å svare på når noe nytt er opprettet, og det er det tabellen i oppgave 4 krever.

> **Hva som gis opp:** Lista på siden blir ikke lenger synkronisert med serveren ved oppretting. Hvis noen andre har lagt til eller slettet spill i mellomtiden, vises ikke det før siden lastes på nytt. For en adminside med få samtidige brukere og lagring i minnet er det et lite tap.

Del 2: Endring med PUT /api/games/{id}

Utgangspunktet

Før ombyggingen kunne adminsiden bare endre kategori, via `PUT /api/games/{id}/category`. Navn og antall spillere kunne ikke endres. Endepunktet svarte `204` uten innhold, så siden hentet alle spillene på nytt etterpå. Tabellen i oppgave 4 krever én PUT på selve spillet, med 400 ved ugyldige data og 404 når id-en ikke finnes.

PUT betyr «erstatt ressursen med dette». Klienten sender derfor alle feltene som kan endres. Id-en står i adressen, ikke i innholdet, fordi det er adressen som sier hvilket spill som endres.

Et kall og et svar

```
PUT /api/games/9
{ "name": "Azul Deluxe", "category": "Abstract", "minPlayers": 2, "maxPlayers": 5 }

→ 200 OK   { "id": 9, "name": "Azul Deluxe", "category": "Abstract", "minPlayers": 2,
    "maxPlayers": 5, ... }
```

Steg 1: UpdateGameRequest

Et spill har de samme reglene når det opprettes og når det endres. `UpdateGameRequest` arver derfor fra `CreateGameRequest`, og får med seg feltene, `[Required]`, `[Range]` og `Validate`, som sjekker at maks ikke er lavere enn min. En egen klasse gjør likevel at endepunktet og skjemaet sier tydelig hva de brukes til, og at de to kan skilles senere hvis reglene blir forskjellige.

Client/UpdateGameRequest.cs (ny fil)

```
namespace AK2.Client
{
    // The body of PUT /api/games/{id}. The id is in the address, not in the body.
    // Inherits the fields and validation from CreateGameRequest, since a game has the same
    rules when created and when changed
    public class UpdateGameRequest : CreateGameRequest
    {
    }
}
```

Steg 2: Lagringen oppdaterer og returnerer spillet

Metoden følger samme mønster som `UpdateCategoryAsync`, men oppdaterer alle de redigerbare feltene. Den returnerer det oppdaterte spillet, eller `null` når id-en ikke finnes. `null` blir til 404 i endepunktet. Id, terningkast og antall vurderinger røres ikke.

Data/GamesStorageAndQuery.cs

```
// Update the whole game. Returns the updated game, or null when the id does not exist
public async Task<BoardGame?> UpdateGameAsync(int id, string name, int minPlayers, int
    maxPlayers, string category)
{
    var game = boardGames.FirstOrDefault(g => g.Id == id);

    if (game is null)
        return null;

    game.Name = name;
    game.MinPlayers = minPlayers;
    game.MaxPlayers = maxPlayers;
    game.Category = category;

    return game;
}
```

Steg 3: Felles validering i servicen

Valideringen fra `CreateGameAsync` er flyttet ut i `ValidateGame`, som både oppretting og endring kaller. Da kan reglene ikke skli fra hverandre, for eksempel ved at en ny regel bare legges inn det ene stedet. Sjekken av at MinPlayers er minst 1 er lagt til her, fordi den står i kravene og bare fantes i skjemaet på klienten. Valideringen på serveren er den som faktisk beskytter dataene, fordi API-et kan kalles uten skjemaet.

ServerServices/ServerGamesService.cs

```
// The same rules apply when a game is created and when it is changed, so the validation is
    written once
private static void ValidateGame(CreateGameRequest request)
{
    if (string.IsNullOrWhiteSpace(request.Name))
    {
        // Throw returns automaticly
        throw new ArgumentException("Navn kan ikke være tomt");
    }

    if (string.IsNullOrWhiteSpace(request.Category))
    {
        throw new ArgumentException("Kategori kan ikke være tom");
    }

    if (request.MinPlayers < 1)
    {
        throw new ArgumentException("Minimum antall spillere må være minst 1");
    }

    if (request.MinPlayers > request.MaxPlayers)
    {
        throw new ArgumentException("Minimum antall spiller kan ikke være større enn maksimum");
    }
}
```

ServerServices/ServerGamesService.cs

```
// Updates the whole game. Returns null when the id does not exist
public async Task<GameDTO?> UpdateGameAsync(int id, UpdateGameRequest request)
{
    ValidateGame(request);

    var game = await query.UpdateGameAsync(id, request.Name, request.MinPlayers, request.
    MaxPlayers, request.Category);
    return game is null ? null : ToDto(game);
}
```

Steg 4: Endepunktet

Endepunktet gir de tre svarene tabellen krever. Det svarer 200 med spillet i stedet for 204, slik at klienten får det oppdaterte spillet og kan bytte det ut i lista. Begge er lov i tabellen. `NotFound` har innhold, slik som DELETE og de andre PUT-ene, slik at `UseStatusCodePagesWithReExecute` ikke gjør svaret om til en HTML-side.

Program.cs

```
// Updates the whole game. 200 with the updated game, 400 on invalid data, 404 when the id does
    not exist
gamesApi.MapPut("{id:int}", async (int id, UpdateGameRequest request, ServerGamesService
    gamesService) =>
{
    try
    {
        var game = await gamesService.UpdateGameAsync(id, request);

        return game is null ? Results.NotFound(new { error = "Spillet finnes ikke" }) : Results.
    Ok(game);
    }

    catch (ArgumentException exception)
    {
        return Results.BadRequest(new
        {
            error = exception.Message
        });
```

Steg 5: GameApiClient

Klienten følger samme mønster som `GetGameByIdAsync`: 404 gir `null`, andre feil kaster unntak, og et vellykket svar gjøres om til `BoardGame`.

Client/GameApiClient.cs

```
// Returns the updated game, or null when the id does not exist (404)
public async Task<BoardGame?> UpdateGameAsync(int id, UpdateGameRequest request)
{
    var response = await http.PutAsJsonAsync($"api/games/{id}", request);

    if (response.StatusCode == HttpStatusCode.NotFound)
    {
        return null;
    }

    response.EnsureSuccessStatusCode();

    var dto = await response.Content.ReadFromJsonAsync<GameDTO>()
        ?? throw new HttpRequestException("API-et returnerte ikke spillet");

    return ToBoardGame(dto);
}
```

Steg 6: Endringsskjema på adminsiden

Kolonnene «Ny kategori» og «Lagre ny kategori» er byttet ut med en «Endre»-knapp per rad. Knappen fyller et `EditForm` med verdiene til spillet. Skjemaet bruker `DataAnnotationsValidator`, så de samme reglene som ved oppretting sjekkes før noe sendes.

Components/Pages/AdminGames.razor: skjemaet

```
@* Changes an existing game. Shown when "Endre" is clicked in the table *@
@if (editingGameId is not null)
{
    <h4>Endre spill (Id @editingGameId)</h4>

    <EditForm Model="EditGame" OnValidSubmit="SaveEdit">

        <DataAnnotationsValidator />
        <ValidationSummary />

        <label>Navn: <InputText @bind-Value="EditGame.Name" /></label>

        <label>Kategori: <InputText @bind-Value="EditGame.Category" /></label>

        <label>Minimum spillere: <InputNumber @bind-Value="EditGame.MinPlayers" step="1" /></
    label>

        <label>Maksimum spillere <InputNumber @bind-Value="EditGame.MaxPlayers" step="1" /></
    label>

        <button type="submit">Lagre endringer</button>
        <button type="button" @onclick="CancelEdit">Avbryt</button>
    </EditForm>
    <br />
}
```

Components/Pages/AdminGames.razor: knappen i tabellen

```
<td><button @onclick="() => StartEdit(game)">Endre</button></td>
```

Når skjemaet lagres, byttes bare det endrede spillet ut i lista med `Select`. Alle andre rader beholdes som de er. Svarer API-et 404, er spillet slettet i mellomtiden. Da fjernes det også fra lista, og siden viser en melding.

Components/Pages/AdminGames.razor: koden

```
// The id of the game being changed. Null when the edit form is hidden
private int? editingGameId;
private UpdateGameRequest EditGame { get; set; } = new();

// Fills the edit form with the current values of the game
private void StartEdit(BoardGame game)
{
    editingGameId = game.Id;
    EditGame = new UpdateGameRequest
    {
        Name = game.Name,
        Category = game.Category,
        MinPlayers = game.MinPlayers,
        MaxPlayers = game.MaxPlayers
    };
}

private void CancelEdit() => editingGameId = null;

private async Task SaveEdit()
{
    if (editingGameId is not int id)
        return;

    try
    {
        var updatedGame = await ApiClient.UpdateGameAsync(id, EditGame);

        editingGameId = null;

        if (updatedGame is null)
        {
            // The game was deleted in the meantime, so it is removed from the list as well
            games = games?.Where(g => g.Id != id).ToList();
            error = "Spillet finnes ikke lenger";
            return;
        }

        // Replaces only the changed game in the list, instead of fetching all games again
        games = games?.Select(g => g.Id == id ? updatedGame : g).ToList();
        error = null;
    }

    catch
    {
        error = "Kunne ikke oppdatere spillet";
    }
}
```

Fordelen med endringen

- **Hele spillet kan endres.** Navn, kategori og antall spillere kan endres i ett skjema, og ikke bare kategorien. Det er det «endre» i kravene for `/admin/games` dekker.
- **Ett kall i stedet for to.** Siden bytter ut ett spill i lista, i stedet for å sende en PUT og deretter hente hele samlingen med GET.
- **Skjemaet er fylt ut på forhånd.** Brukeren endrer bare det som skal endres. Resten sendes med uendret, slik PUT forventer.
- **Samme regler begge steder.** `UpdateGameRequest` arver valideringen i skjemaet, og `ValidateGame` deles på serveren. En regel som legges til, gjelder både oppretting og endring.
- **Kontrollert håndtering av et slettet spill.** 404 fører til at raden fjernes og at det vises en melding, i stedet for at en rad som ikke finnes lenger, blir stående.

- **Svaret viser det lagrede spillet.** Lista viser det serveren faktisk har lagret, ikke det brukeren skrev i skjemaet.

Testet mot API-et

Appen er bygget og startet, og endepunktene er kalt direkte. Dette er svarene:

Kall	Svar
`POST` med gyldig spill (Azul)	`201 Created`, `Location: /api/games/9`, spillet med id 9
`POST` med tomt navn	`400` «Navn kan ikke være tomt»
`POST` med MinPlayers 0	`400` «Minimum antall spillere må være minst 1»
`PUT /api/games/9` med nytt navn og maks 5	`200 OK` med det oppdaterte spillet
`PUT /api/games/9` med min 5 og maks 2	`400` «Minimum antall spiller kan ikke være større enn maksimum»
`PUT /api/games/9999`	`404 { "error": "Spillet finnes ikke" }`
`GET /api/games/9` etter PUT	Det oppdaterte spillet
En ukjent side, `/finnes-ikke`	Fortsatt «Fant ikke»-siden med status 404

Ikke testet her: klikk i adminsiden i nettleseren. Koden kompilerer, men siden må prøves i Visual Studio. Testprosjektet er heller ikke kjørt etter endringen.

Sjekk i nettleseren	Forventet
Opprett et spill på `/admin/games`	Spillet dukker opp nederst i tabellen med neste ledige id, uten «Laster spill...»
Klikk «Endre» på en rad	Skjemaet vises med spillets verdier fylt ut
Endre navn og lagre	Bare den raden endres. Skjemaet forsvinner
Sett maks lavere enn min og lagre	Valideringsmelding i skjemaet. Ingenting sendes
«Avbryt» i endringsskjemaet	Skjemaet forsvinner. Ingenting endres
Slett et spill, og prøv å lagre en endring på det fra en annen fane	«Spillet finnes ikke lenger», og raden forsvinner

Det som står igjen

- **Spilletid.** Kravet «spilletiden er over 0» kan ikke valideres før spillet har en egenskap for spilletid i minutter. Når den legges til, må den med i `BoardGame`, `GameDTO`, `ToDto`, `ToBoardGame`, `CreateGameRequest` (og dermed `UpdateGameRequest`), `ValidateGame`, `CreateGameAsync` og `UpdateGameAsync` i lagringen, `DummyGames`, og i begge skjemaene og `StartEdit` på adminsiden.
- **Sletting** henter fortsatt alle spillene på nytt med `LoadGames()`. Samme idé kan brukes der: fjern spillet fra lista når `DeleteGameAsync` svarer `true`.
- `PUT /api/games/{id}/category` og `UpdateCategoryAsync` i `GameApiClient` og servicen brukes ikke lenger av adminsiden. Om de skal fjernes eller beholdes, er et eget valg.
- **Terningkastet** er ikke med i `UpdateGameRequest`. Det endres fortsatt via `PUT /api/games/{id}/rating`, der 1–6 valideres.